

```
$ curl /users/123 | usecase | repository | db
```

# Clean Architecture

// 分層，但不互相認識

**Controller**

服務生

**Usecase**

主廚

**Repository**

倉管

**Presenter**

打包員

```
$ cat blueprint.md
```

# 今日路線

## 01 / 為什麼

全黏在一起的程式怎麼讓你崩潰

## 02 / 四個角色

服務生、主廚、倉管、打包員

## 03 / 實作

TypeScript 完整範例 + 依賴注入

## 04 / 驗收

換 DB、換格式、加快取——各改哪層？

核心規則只有一條：你只跟上一層說話，下層的細節不關你的事。

PART 01

# 為什麼需要它？

先看一段讓人崩潰的程式

# 全黏在一起

```
1 // UI 直接連資料庫
2 app.get('/users', (req, res) => {
3   const users = db.query('SELECT * FROM users');
4   res.json(users.map(u => ({
5     name: u.name,
6     email: u.email,
7     createdAt: u.created_at.format('YYYY-MM-DD')
8   })));
9 });
```

INCOMING REQUESTS

老闆：「我們要換 GraphQL！」

→ 你崩潰（路由、格式全重寫）

老闆：「加個快取！」

→ 改 10 個地方

老闆：「換 MongoDB！」

→ SQL 散落各處，找都找不完

# 分層之後：改「怎麼做」，不動「做什麼」

換資料庫？

只改 Repository

換 API 格式？

只改 Presenter

換前端框架？

只改 Controller

INVARIANT

商業邏輯 (Usecase) 永遠不動 — 它是你的核心資產，活得比任何框架久。

PART 02

# 四個角色

一間餐廳就講完了

# 餐廳編制

## Controller = 服務生

接單、把參數轉成程式能用的格式、傳話給廚房。**不能寫商業邏輯！**

## Usecase = 主廚

醃多久、火候多大——商業邏輯的家。不知道資料庫長怎樣、不知道前端要 JSON 還是 XML

## Repository = 倉管

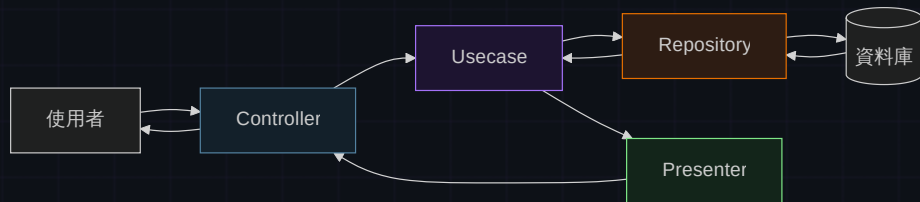
主廚喊「拿雞蛋」就去拿。從 MySQL、MongoDB 還是 API 拿？主廚不關心

## Presenter = 打包員

JSON 盒子還是 XML 盒子？只管怎麼呈現，不管資料哪來

重點：廚房不知道菜市場在哪。大家只管自己的事。

# 請求流程：資料只往內流



## 去程

UI → Usecase → Repository → DB

## 回程

DB → Repository → Usecase → Presenter → UI，永遠不反向

PART 03

# TypeScript 實作

「取得使用者」完整走一遍

# Repository : 先定契約

```
1  export interface UserRepository {
2    getUser(id: number): Promise<User | null>;
3  }
4
5  // 實作：用 MySQL — Usecase 完全不知道
6  export class MySQLUserRepository
7    implements UserRepository {
8    async getUser(id: number) {
9      const rows = await db.query(
10        'SELECT * FROM users WHERE id = ?', [id]);
11      return rows[0] ?? null;
12    }
13  }
```

# Usecase : 商業邏輯的家

```
1  export class GetUserUseCase {
2    constructor(
3      private userRepository: UserRepository
4    ) {}
5
6    async execute(id: number) {
7      const user =
8        await this.userRepository.getUser(id);
9      if (!user) return null;
10
11      // 真正的商業邏輯：30 天未登入視為無效
12      if (user.createdAt < thirtyDaysAgo()) {
13        return null;
14      }
15      return user;
16    }
17  }
```

依賴介面，不依賴實作

建構子注入 UserRepository —— MySQL 還是 Mongo？不知道也不需要知道

# Presenter 與 Controller

```
1 // Presenter : 只管怎麼呈現
2 export class UserPresenter {
3   static toResponse(user: User | null) {
4     if (!user) return { error: 'User not found' };
5     return {
6       id: user.id,
7       name: user.name,
8       created_at: user.createdAt.toISOString(),
9     };
10  }
11 }
```

# 組裝端：依賴注入

```
1 // routes/user.ts — 具體選擇集中在這
2 const repo = new MySQLUserRepository();
3 const usecase = new GetUserUseCase(repo);
4 const controller = new UserController(usecase);
5
6 router.get('/users/:id', controller.getUser);
```

## COMPOSITION ROOT

Controller 不知道資料庫是什麼、Usecase 不知道前端要什麼 — 每層都只跟介面說話。

PART 04

# 驗收時間

老闆的三個需求，各改哪一層？

# 三個需求，三次「只改一層」

## 「換成 MongoDB！」

新增 MongoDBUserRepository 實作介面，組裝端換一行 new。其他層零改動。

## 「改輸出 XML！」

只改 Presenter 的 toResponse()。其他層零改動。

## 「查詢太慢，加快取！」

CacheUserRepository 包住 MySQLUserRepository（裝飾器模式），先查快取再查 DB。Usecase 完全沒動。

\$ 每個需求都只動一層 — 這就是分層的投資報酬率。

## CHECKPOINT\_01

使用者抱怨查詢太慢，你決定加快取（Cache）。該改哪一層？

A Usecase：在商業邏輯裡加一段查快取的程式

B Controller：收到請求先看看快取有沒有

C Repository：包一層 CacheUserRepository，先查快取再查 DB

# FAQ 快問快答

## 這不就是 MVC 嗎？

MVC 常把邏輯與存取黏在一起；Clean Architecture 是把「做什麼」和「怎麼做」分家。

## 寫這麼多檔案不累嗎？

寫 10 個檔案累，還是改 100 個地方時崩潰累？現在省 1 小時，未來用 100 小時還。

## 小工具也要這樣？

不用。但希望它活過 2 年的東西，就值得分層。

## Repository 要寫介面嗎？

要。介面讓 Usecase 不知道你用什麼資料庫——明天才換得掉。

# 今日重點

## 一條規則

資料流只能往內，不能往外

## 四個角色

服務生、主廚、倉管、打包員——各管各的

## 依賴介面

具體選擇集中在組裝端（依賴注入）

## 驗收標準

換 DB、換格式、加快取——各只改一層

分層，但不互相認識 —— 商業邏輯永遠不動。

# Layers separated.

下一步：Spring Boot × DDD，看同樣的思想在後端怎麼落地

```
$ curl /users/123
```

```
controller → usecase → repository → presenter
```